



Faculty of Engineering and Technology
Electrical and Computer Engineering Department

DESIGN VERIFICATION (ENCS 5337)

Section: 2

Instructor: Elias Khalil

BIRD Project
Functional Coverage and Verification Report
Birzeit Integrated Router Design

Group Members:

Ibraheem Sleet	ID: 1220200
Aws Hammad	ID: 1221697
Sari Abdalghani	ID: 1220982
Mohammad Shamasneh	ID: 1220092

Date: 19/6/2026

Contents

1.	Objective	2
2.	Test Environment Summary	2
3.	BIRD Protocol Summary	2
4.	Verification Environment	3
5.	Functional Coverage Results	4
5.1.	Testbench Covergroup Scores	4
5.2.	Merged Coverage Summary	4
6.	Analysis	5
6.1.	Backpressure Coverage	5
6.2.	Configuration Field Coverage	5
6.3.	Reset Coverage	5
6.4.	Remote Reassembly Coverage	5
6.5.	Drop Reason Coverage	6
7.	Example Bug Evidence	6
8.	Test Plan Summary	6

1. Objective

This report presents the functional coverage results obtained from the verification environment of the BIRD protocol testbench. BIRD stands for Birzeit Integrated Router Design. The purpose of this verification phase is to evaluate how well the testbench exercises the main functional scenarios of the router, including configuration field values, drop conditions, remote reassembly, backpressure behavior, and reset scenarios.

The coverage results are used to identify which parts of the verification plan have already been exercised and which parts still require additional directed or randomized test cases. The report also explains why some tests may pass even when the DUT is bugged, because a passing test only proves that the specific scenario covered by that test works.

2. Test Environment Summary

The coverage data was collected using Synopsys VCS and the Unified Report Generator (URG). The simulation was run using the BIRD testbench, which includes a driver, input monitor, local output monitor, remote output monitor, scoreboard, and functional coverage collector.

Item	Value
Tool Version	T-2022.06-SP2
Simulator	Synopsys VCS
Coverage Tool	URG / Verdi Coverage Report
Number of Tests	17 tests
Total Functional Coverage Score	93.33%

Table 1: Test environment and merged coverage run summary.

The testbench compares the DUT output against the expected behavior using a scoreboard. The scoreboard is based on the BIRD specification and not on the RTL implementation. This allows the environment to detect mismatches between the provided DUT and the required behavior.

3. BIRD Protocol Summary

The BIRD design is a packet-based router with one input interface and two output interfaces: local and remote. The input stream consists of payload bytes followed by a 16-bit CRC. A 32-bit

configuration field, `cfg`, is provided as a side-band signal and is sampled on the same clock cycle as the first payload byte of each fragment.

The important configuration fields are:

Bits	Field	Description
<code>cfg[0]</code>	Traffic Type	0 = Local traffic, 1 = Remote traffic
<code>cfg[7:1]</code>	Reserved	Must be zero
<code>cfg[15:8]</code>	Payload Length	Number of payload bytes, from 1 to 255
<code>cfg[20:16]</code>	Fragment Number	Fragment index used for remote reassembly
<code>cfg[23:21]</code>	Reserved	Must be zero
<code>cfg[28:24]</code>	Sequence Number	Packet identifier; zero is invalid
<code>cfg[31:29]</code>	Reserved	Must be zero

Table 2: BIRD configuration field summary.

Local traffic is forwarded directly to the local output. Remote traffic may arrive in fragments, and the DUT must accumulate fragments, reorder them using `FRAG_NUM`, merge the payloads, regenerate the CRC16 over the merged payload, and transmit the result on the 32-bit remote output.

4. Verification Environment

The verification environment was built using a structured SystemVerilog testbench. The main components are:

- **Transaction:** Represents one BIRD fragment, including traffic type, payload length, fragment number, sequence number, payload bytes, and CRC.
- **Sequences:** Generate local packets, remote packets, invalid packets, backpressure scenarios, and reset scenarios.
- **Driver:** Drives the input interface using the valid/ready protocol.
- **Input Monitor:** Reconstructs input fragments from the input bus.
- **Local Monitor:** Captures local output bytes.
- **Remote Monitor:** Captures remote output words.

- **Scoreboard:** Builds the expected local and remote output streams and compares them with the DUT output.
- **Coverage Collector:** Samples configuration fields, drop reasons, remote reassembly scenarios, backpressure behavior, and reset scenarios.

The scoreboard was designed to catch both data mismatches and protocol errors. For example, it can detect missing local bytes, unexpected local bytes, remote word mismatches, incorrect drop counter values, and incorrect packet reassembly.

5. Functional Coverage Results

5.1. Testbench Covergroup Scores

The functional coverage summary contains five main covergroups. Each covergroup targets one important verification area. After running and merging 17 tests, the total functional coverage score increased to 93.33%. This shows that the verification environment now covers most of the important BIRD protocol scenarios.

All covergroups belong to the `bird_pkg::bird_coverage` class. The table below shows the individual covergroup scores.

Covergroup	Score (%)	Goal (%)	Weight
cg_bp_cg	75.00	100	1
cg_cfg_cg	91.67	100	1
cg_reset_cg	100.00	100	1
cg_remote_cg	100.00	100	1
cg_drop_cg	100.00	100	1

Table 3: Merged functional coverage score per covergroup.

5.2. Merged Coverage Summary

The merged URG dashboard reports the following final functional coverage result:

```

1 Total Coverage Summary
2   Score : 93.33 %
3   Group : 93.33 %
4
5 Total Groups Coverage Summary

```

```

6   cg_bp_cg      : 75.00 %
7   cg_cfg_cg     : 91.67 %
8   cg_reset_cg   : 100.00 %
9   cg_remote_cg  : 100.00 %
10  cg_drop_cg    : 100.00 %
11
12  Number of tests: 17

```

Listing 1: Merged functional coverage summary

The result improved significantly because multiple tests were executed and their coverage databases were merged into one final report.

6. Analysis

6.1. Backpressure Coverage

Backpressure coverage achieved 75.00%. This means the testbench successfully exercised most local and remote ready/valid stall scenarios. The remaining uncovered bins likely correspond to specific stall timing combinations that were not hit during the current regression.

6.2. Configuration Field Coverage

Configuration field coverage achieved 91.67%. This is a strong result and shows that the tests covered a wide range of legal and illegal configuration values, including traffic type, payload length, sequence number, fragment number, and reserved field cases.

6.3. Reset Coverage

Reset coverage reached 100.00%. This means the regression successfully covered reset scenarios such as reset during idle state, reset during packet input, and reset during active verification behavior.

6.4. Remote Reassembly Coverage

Remote reassembly coverage reached 100.00%. This confirms that the testbench exercised important remote packet scenarios, including single-fragment packets, multi-fragment packets, out-of-order fragments, missing fragments, and remote packet reassembly behavior.

6.5. Drop Reason Coverage

Drop reason coverage reached 100.00%. This means all defined packet drop scenarios were triggered during regression, including invalid sequence number, invalid fragment number, zero payload length, reserved-bit violations, mismatched sequence numbers, and missing remote fragments.

Overall, the merged coverage score of 93.33% is a strong verification result. The only major remaining gap is the backpressure covergroup, which reached 75.00%. Additional randomized ready/valid stall tests can be added to improve it further.

7. Example Bug Evidence

A passing test does not mean the DUT is bug-free. For example, `test_local_basic` can pass on the old bugged DUT because it only uses local packets with `SEQ_NUM=1` and `FRAG_NUM=1`. This does not expose the bug where the DUT incorrectly drops local packets with non-one sequence numbers.

A stronger test is `test_local_seqnum_variation`, which sends valid local packets using different nonzero sequence numbers. According to the specification, local `SEQ_NUM` should not affect routing. Therefore, these packets should still be forwarded to the local output.

```
1 [SCOREBOARD] Missing LOCAL byte: expected 0x40 but DUT did not output
   it
2 [SCOREBOARD] drop_cnt mismatch: expected=0 actual=6
3 RESULT: FAIL
```

Listing 2: Example scoreboard failure exposing local `SEQ_NUM` bug

This failure indicates that the bugged DUT incorrectly dropped valid local packets when the sequence number was not equal to one.

8. Test Plan Summary

The following table summarizes the main tests planned for the BIRD verification project.

Test Name	Category	Purpose
<code>test_local_basic</code>	Local Traffic	Verify that valid local packets are forwarded to the local output with payload and CRC unchanged.
<code>test_local_seqnum_variation</code>	Local Traffic	Verify that local sequence number values do not affect local routing as long as they are nonzero.

Test Name	Category	Purpose
test_remote_single_fragment	Remote Traffic	Verify that one remote fragment is processed and transmitted on the remote output.
test_remote_out_of_order	Remote Traffic	Verify that remote fragments are reordered using FRAG_NUM.
test_backpressure_remote	Backpressure	Verify that data_remote remains stable when remote_vld=1 and remote_rdy=0.
test_drop_seq_zero	Drop Policy	Verify that packets with SEQ_NUM=0 are dropped and counted once.
test_drop_frag_zero	Drop Policy	Verify that packets with FRAG_NUM=0 are dropped and counted once.
test_drop_payload_len_zero	Drop Policy	Verify that zero-length payload packets are dropped and counted once.
test_reset_during_packet	Reset	Verify that reset clears all state and discards packets in progress.
test_full_random_regression	Regression	Run mixed valid, invalid, local, remote, reset, and backpressure scenarios to improve coverage.

Table 4: Summary of planned BIRD verification tests.